

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN
MẠNG MÁY TÍNH

Ứng dụng FTP bảo mật tích hợp quét virus bằng ClamAV Agent

Giáo viên hướng dẫn: Cô Huỳnh Thụy Bảo Trân
Cô Chung Thùy Linh

Sinh viên thực hiện:

Họ tên	Mã số sinh viên	Lớp
Vũ Duy Nhất	24127095	24C11
Nguyễn Nhật Huy	24127398	24C11
Trịnh Khánh Linh	24127437	24C11

TP. Hồ Chí Minh, 2025

THÔNG TIN VỀ ĐỒ ÁN

Mã học phần: CSC10008

Tên học phần: Mạng máy tính

Tên đồ án: Ứng dụng FTP bảo mật tích hợp quét virus bằng ClamAVAgent

Hình thức: Bài tập nhóm

Hình thức nộp bài: Nộp mã nguồn + video demo + báo cáo PDF

Mô tả:

Đồ án xây dựng một hệ thống FTP Client mô phỏng quá trình truyền file an toàn, có tích hợp kiểm tra virus trước khi upload. Hệ thống gồm 3 thành phần: Client (giao diện GUI), Agent (ClamAV Server để quét virus), và FTP Server (FileZilla). Mọi hoạt động truyền file và quét đều sử dụng lập trình socket thuần.

Chức năng chính:

1. Gửi file từ client đến agent để kiểm tra virus trước khi upload.
2. Chỉ upload file nếu được xác nhận là an toàn.
3. Hỗ trợ các lệnh FTP cơ bản như ls, cd, put, get, delete, rename,...

Giảng viên phụ trách: Cô Huỳnh Thụy Bảo Trân, Cô Chung Thùy Linh

Mục lục

1	Tổng quan hệ thống	4
2	Kiến trúc hệ thống	4
3	Minh họa phiên làm việc	6
4	Vấn đề gặp phải	10
4.1	Lựa chọn ngôn ngữ để bắt đầu xây dựng	10
4.2	Quá trình xây dựng	10
4.3	Giao diện chương trình	10
4.4	Xử lý luồng chương trình	10
5	Các tính năng của chương trình phục vụ người dùng	11
5.1	Các thao tác với tập tin và thư mục	11
5.2	Upload và Download	11
5.3	Quản lý phiên làm việc	12
6	Hoạt động nhóm	13
6.1	Đánh giá thành viên	13
6.2	Biên bản họp nhóm	13
6.2.1	Họp nhóm ngày 24/06	13
6.2.2	Họp nhóm ngày 26/06	14

§1 Tổng quan hệ thống

- Mục tiêu: Truyền file lên FTP Server một cách an toàn bằng cách quét virus trước khi upload.
- Mô hình có 3 thành phần:
 - + **FTP Client:** Được lập trình bằng ngôn ngữ Python, có giao diện người dùng trực quan sử dụng thư viện Tkinter. Người dùng có thể thực hiện các thao tác trên tập tin và thư mục như đăng nhập, đổi tên, xóa, tải lên/tải xuống, xem thư mục hiện tại,...
 - + **ClamAVAgent:** Một trung gian có nhiệm vụ tiếp nhận file từ client và thực hiện quét virus bằng công cụ ClamAV. Nếu file được xác nhận an toàn (không nhiễm virus), client mới được phép upload lên FTP Server.
 - + **FTP Server:** Sử dụng FileZilla để lưu trữ các file đã được kiểm duyệt và xử lý từ client.

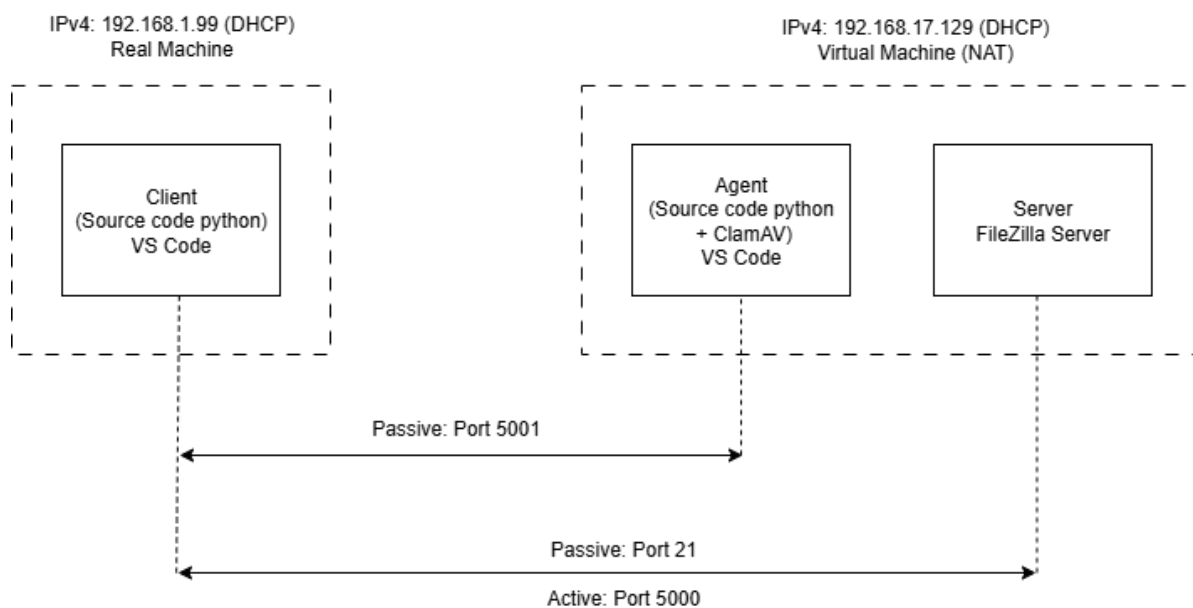
Giao tiếp giữa các thành phần được thực hiện thông qua **socket TCP thuần**, không sử dụng các thư viện FTP cấp cao. Đặc biệt, trước mỗi lần upload file lên FTP Server, file sẽ được client gửi trước đến Agent để kiểm tra an toàn, đảm bảo rằng không có tập tin độc hại nào được lưu trữ trên hệ thống.

Ứng dụng client sử dụng thiết kế đa luồng, trong đó mỗi thao tác (ví dụ: gửi lệnh, nhận phản hồi, truyền file) đều được thực thi trên một **thread riêng** nhằm tránh xung đột với **thread chính điều khiển giao diện GUI**, đảm bảo tính ổn định và phản hồi nhanh của chương trình.

- Luồng hoạt động chính:
 1. Client gửi file đến Agent.
 2. Agent quét virus.
 3. Nếu sạch → file được upload lên FTP Server.
 4. Nếu nhiễm virus → quá trình bị hủy và cảnh báo hiển thị.

§2 Kiến trúc hệ thống

Hình dưới mô tả chi tiết kiến trúc triển khai hệ thống trên môi trường thực tế của nhóm. Hệ thống được chia làm hai phần chính: một máy thật (real machine) và một máy ảo (virtual machine).



Hình 1: Sơ đồ hoạt động

- **Máy thật (Client):**

- Địa chỉ IPv4: 192.168.1.99, được cấp phát qua DHCP nên sẽ thay đổi tùy theo thời điểm kết nối wifi nên cần kiểm tra lại trước khi chạy hệ thống (vào Command Prompt nhập lệnh `ipconfig`).
- Chạy ứng dụng FTP Client được viết bằng Python và thiết kế giao diện bằng thư viện Tkinter.
- Chạy trên môi trường phát triển VS Code.

- **Máy ảo (Agent + Server):**

- Địa chỉ IPv4: 192.168.17.129, được cấp phát qua DHCP nên sẽ thay đổi tùy theo thời điểm kết nối wifi nên cần kiểm tra lại trước khi chạy hệ thống (vào Command Prompt nhập lệnh `ipconfig`).
- Chạy Agent (viết bằng Python), tích hợp phần mềm ClamAV để quét virus.
- Đồng thời trên máy ảo cũng cài đặt FileZilla Server để tiếp nhận các file sạch sau khi quét.

- **Kết nối mạng giữa các thành phần:**

- Client kết nối đến Agent thông qua **Port 5001 (Passive)** để gửi file cần quét.
- Client kết nối đến FileZilla Server qua cổng **Port 21 (Passive)** để truyền lệnh và file.
- Trong chế độ Active FTP, client mở **Port 5000** để chờ server kết nối lại.

(Lưu ý cấu hình tường lửa trước khi chạy hệ thống)

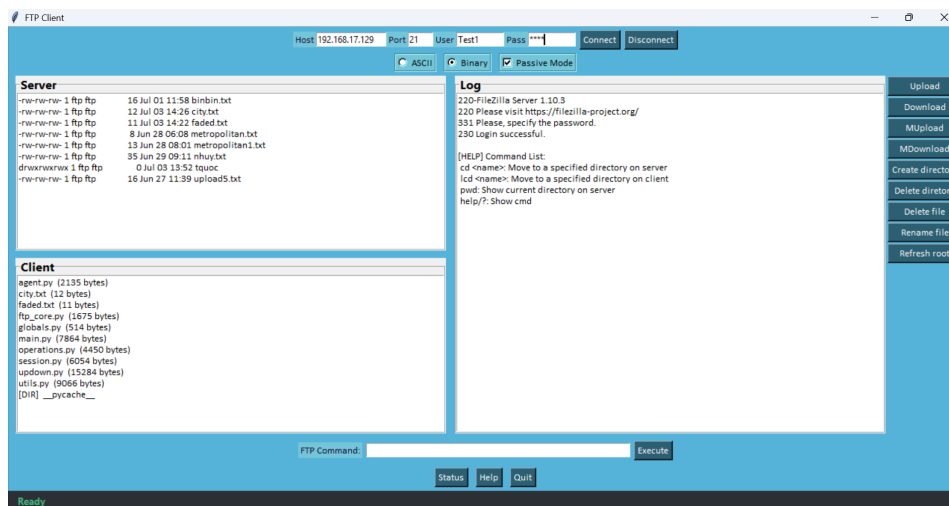
- **Quy trình hoạt động của thao tác put:**

1. Người dùng dùng lệnh `put` trên FTP Client để gửi file.
2. File được gửi đến ClamAV để quét virus.

3. Nếu kết quả là **OK**, client sẽ tiến hành upload file lên FTP Server.
4. Nếu kết quả là **INFECTED**, quá trình upload bị hủy.

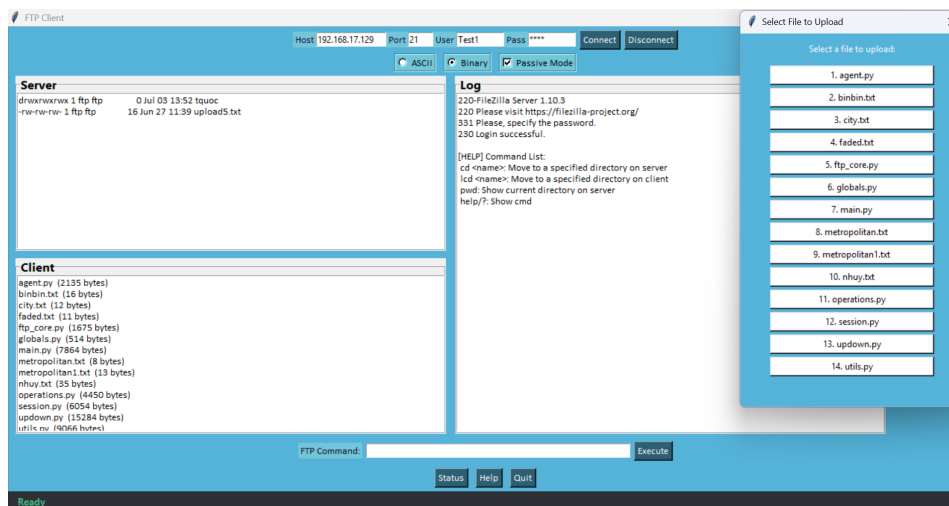
§3 Minh họa phiên làm việc

1. Đăng nhập vào tài khoản đã cấu hình trên FTP Server

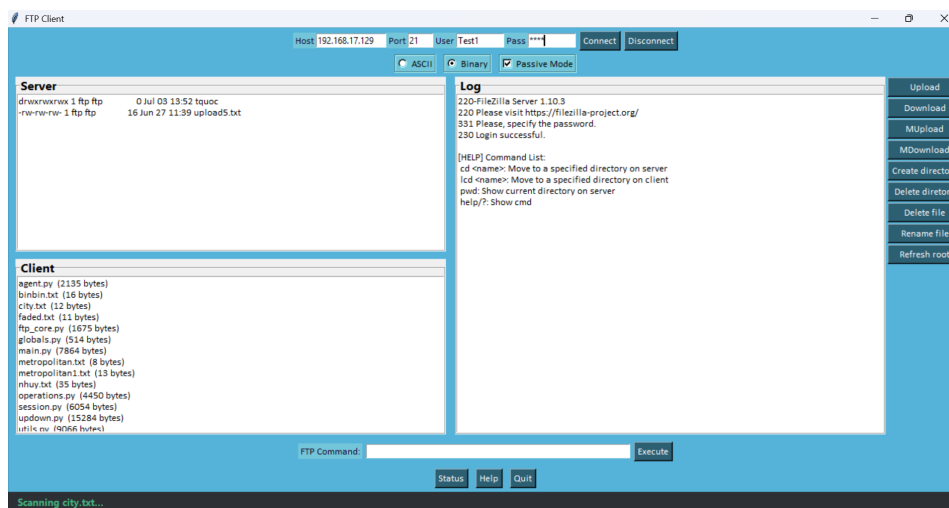


Hình 2: Đăng nhập thành công và hiển thị các lệnh có sẵn

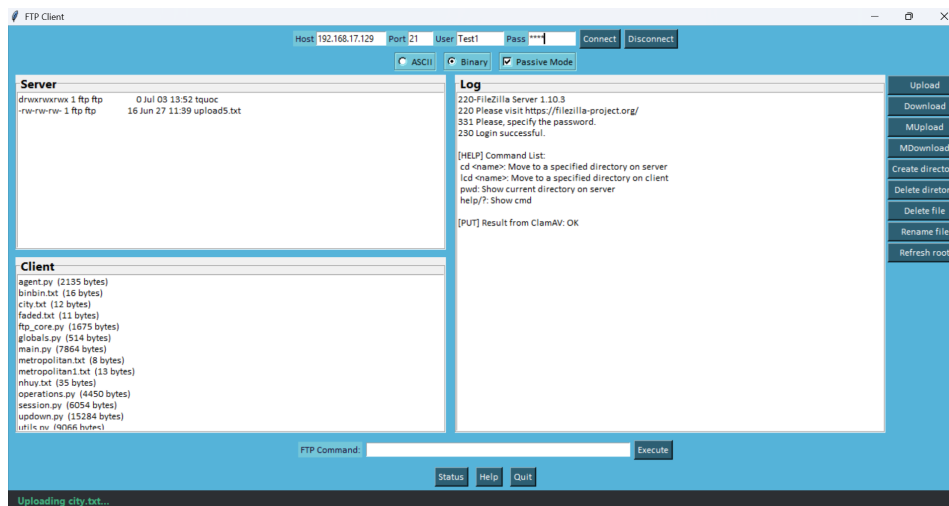
2. Quét virus và tải thành công file lên server



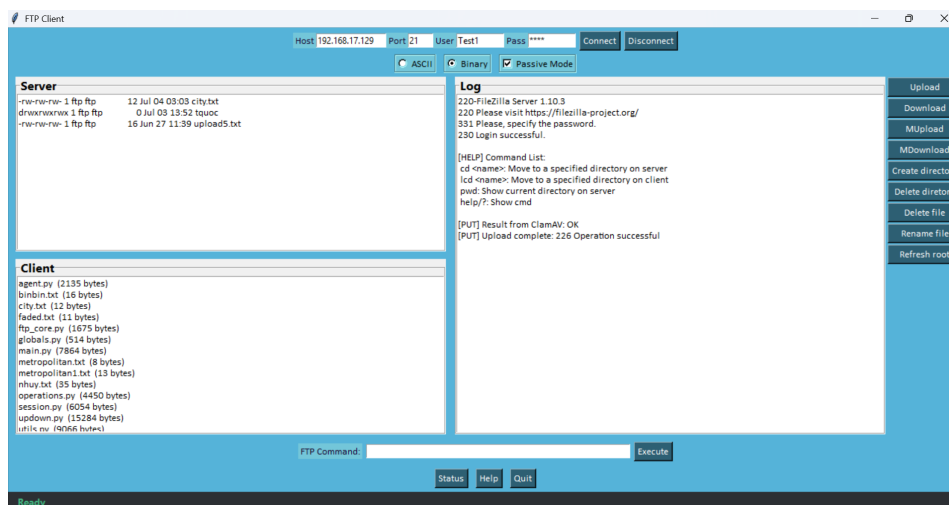
Hình 3: Chọn file để upload lên server



Hình 4: Thanh trạng thái hiển thị đang quét file

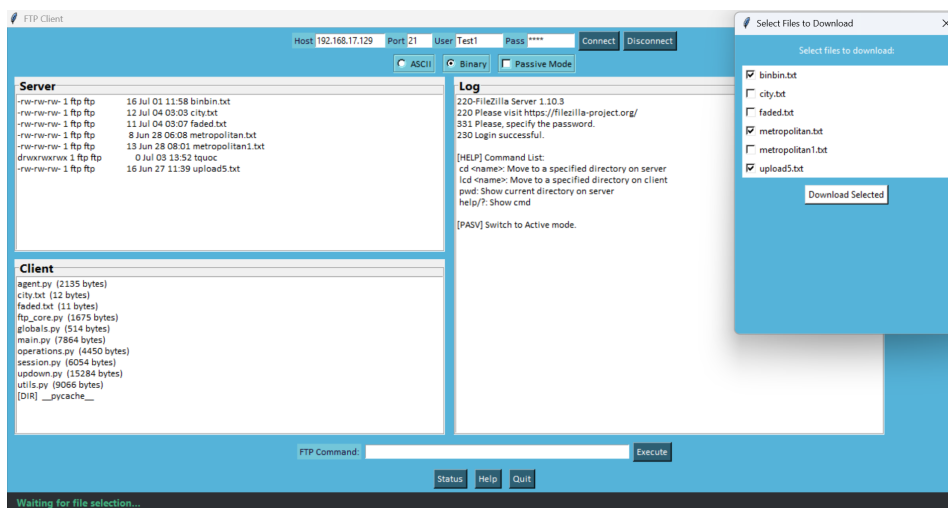


Hình 5: Upload file sau khi quét xong

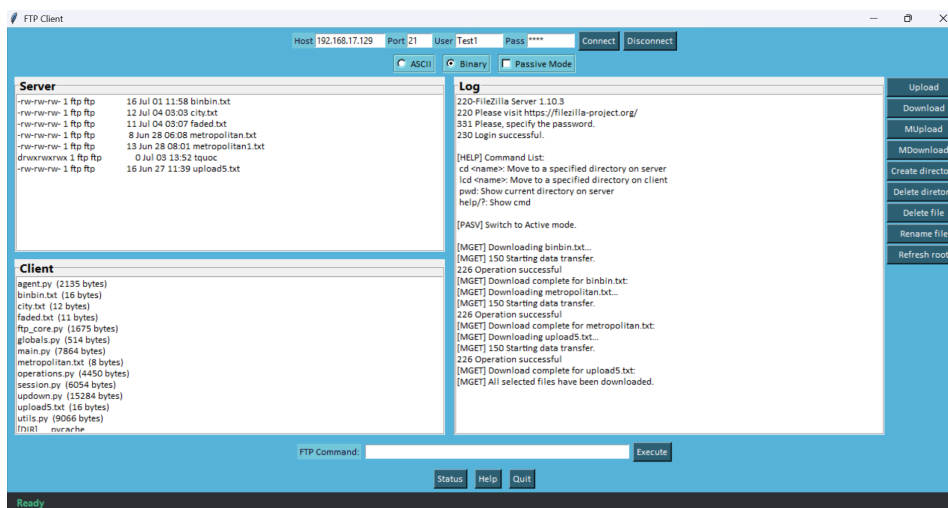


Hình 6: Màn hình hiển thị upload file thành công

3. Tải xuống thành công nhiều file từ server

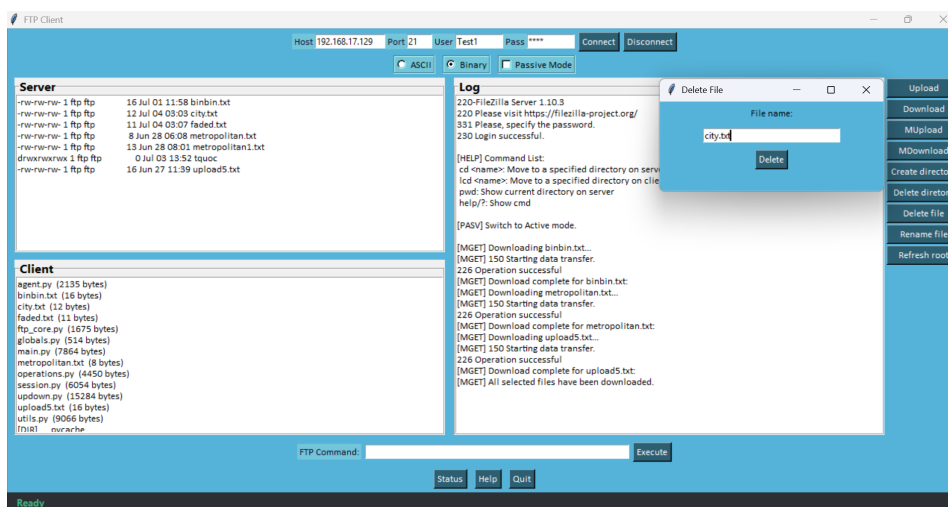


Hình 7: Chọn những file muốn tải xuống

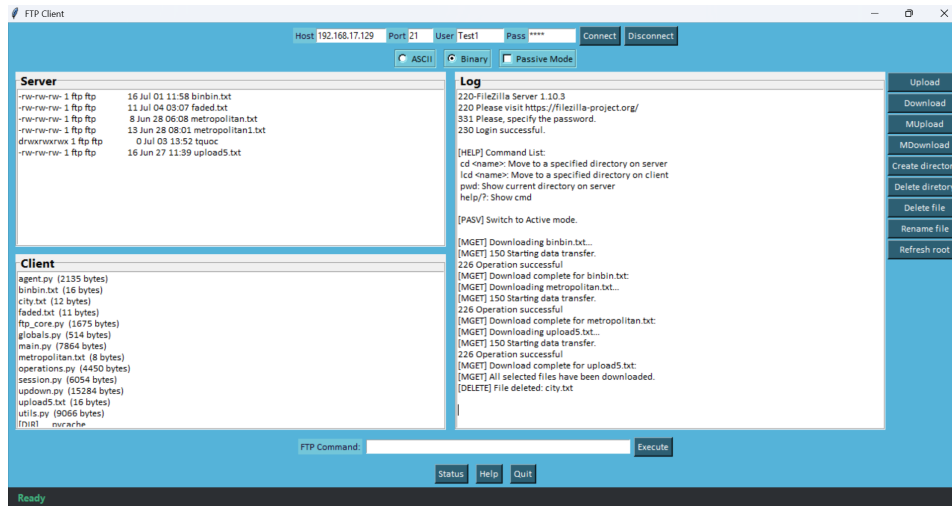


Hình 8: Tải thành công nhiều file từ server

4. Xóa file

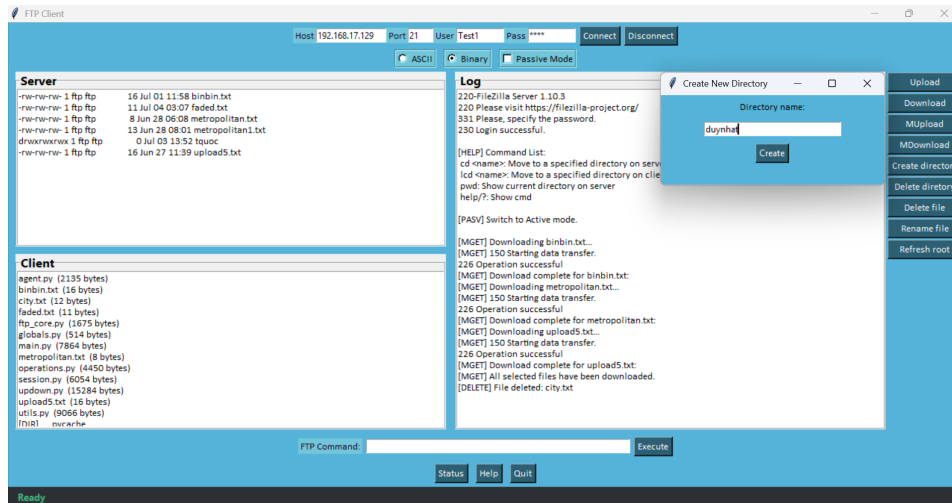


Hình 9: Nhập tên file muốn xóa

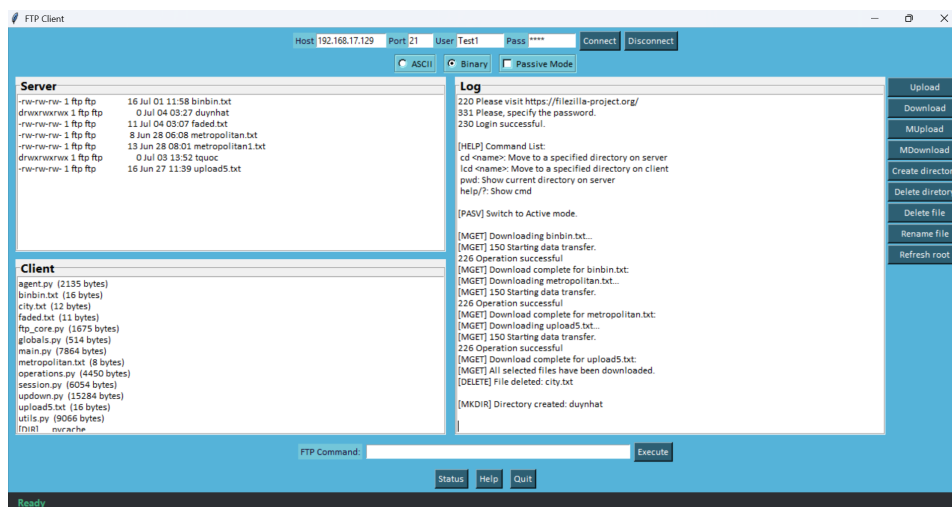


Hình 10: Xóa file thành công

5. Tạo thư mục

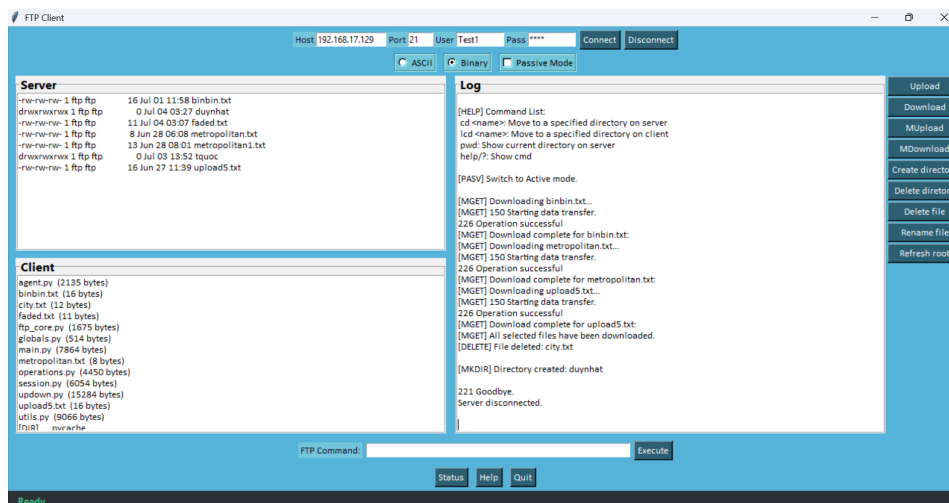


Hình 11: Nhập tên thư mục mới



Hình 12: Tạo thư mục thành công

6. Đăng xuất sau khi làm việc xong



Hình 13: Đăng xuất thành công

§4 Vấn đề gặp phải

§4.1 Lựa chọn ngôn ngữ để bắt đầu xây dựng

Ban đầu nhóm định lựa chọn ngôn ngữ C++ vì đây là ngôn ngữ quen thuộc hơn vì đã được tiếp xúc nhiều nhưng nhóm sớm nhận ra rằng C++ có cú pháp khá phức tạp trong việc lập trình socket. Sau một thời gian tìm hiểu và cân nhắc, nhóm đã quyết định lựa chọn Python làm ngôn ngữ chính để xây dựng chương trình vì cú pháp đơn giản, dễ thử nghiệm và có nhiều thư viện hỗ trợ tốt cho socket và giao diện.

§4.2 Quá trình xây dựng

Vì đây là lần đầu nhóm tiếp xúc với lập trình socket nên gặp nhiều khó khăn trong việc định hướng và bắt đầu. Nhóm đã dành khoảng hai ngày đầu tiên để tìm hiểu kiến thức nền như giao thức FTP, cách hoạt động của FTP server, socket programming,... Sau đó, các thành viên họp lại để chia sẻ kiến thức, thảo luận và phân chia công việc phù hợp. Mỗi khi có vấn đề phát sinh, các thành viên đều trao đổi để cùng giải quyết.

§4.3 Giao diện chương trình

Nhóm đã quyết định sử dụng thư viện Tkinter của Python để xây dựng giao diện người dùng. Tuy nhiên, do chưa có nhiều kinh nghiệm với Tkinter, nhóm gặp một số lỗi liên quan đến việc sắp xếp giao diện (layout) và hiển thị chưa như mong muốn. Để xử lý, nhóm đã debug từng phần tử **frame** một cách cẩn thận, điều chỉnh vị trí và thuộc tính của các thành phần để đảm bảo giao diện giống thiết kế ban đầu.

§4.4 Xử lý luồng chương trình

Trong quá trình phát triển, nhóm nhận thấy việc xử lý luồng là rất quan trọng để đảm bảo chương trình hoạt động mượt mà. Nếu không tách riêng luồng xử lý lệnh FTP ra khỏi luồng giao diện, giao diện sẽ bị treo khi thực thi các tác vụ như tải file hoặc kết nối mạng. Vì vậy, nhóm đã bổ sung thêm một luồng phụ chuyên xử lý lệnh. Đồng thời,

chương trình được thiết kế sao cho chỉ cho phép một luồng xử lý lệnh hoạt động tại một thời điểm nhằm tránh xung đột khi người dùng thao tác nhiều nút liên tiếp.

§5 Các tính năng của chương trình phục vụ người dùng

Hệ thống đã đáp ứng đầy đủ các yêu cầu chức năng được đề ra, bao gồm thao tác với tập tin, quản lý phiên làm việc và đảm bảo an toàn khi upload file. Cụ thể như sau:

§5.1 Các thao tác với tập tin và thư mục

- **ls**: Hiển thị danh sách tập tin và thư mục trên server ngay sau khi kết nối thành công; giao diện tự động cập nhật khi có thay đổi.
- **cd**: Cho phép người dùng nhập tên thư mục và chuyển thư mục trên server thông qua thanh lệnh.
- **lcd**: Di chuyển đến đường dẫn mà người dùng nhập vào ở máy Client (mặc định là ở thư mục chứa chương trình) trên thanh FTP command, sau khi thực thi cửa sổ Client ở giao diện sẽ tự động cập nhật lại.
- **pwd**: người dùng nhập lệnh vào thanh FTP command và chương trình sẽ in ra cửa sổ Log thư mục hiện tại trên server.
- **mkdir, rmdir**: tạo/xóa thư mục theo tên người dùng nhập vào và in kết quả ra cửa sổ Log bằng nút bấm *Create/Delete directory* trên giao diện FTP Client.
- **delete**: Xóa tập tin theo tên được nhập từ người dùng qua nút *Delete file*.
- **rename**: Đổi tên tập tin bằng một tên mới do người dùng nhập vào thông qua nút *Rename file*.

§5.2 Upload và Download

- **get**: Hiển thị hộp thoại chọn file để tải từ server về client, thực hiện qua nút *Download*.
- **put**: Mở hộp thoại chọn file để upload từ client; file được gửi đến Agent để quét bằng ClamAV trước khi đưa lên server.
- **mget**: Cho phép người dùng chọn nhiều file cùng lúc từ server để tải về thông qua nút *MDownload*, tải về từng file người dùng đã chọn.
- **mput**: Cho phép chọn nhiều file từ client, từng file được quét virus rồi upload thông qua nút *MUpload*.
- **prompt**: Tích hợp sẵn với **mput**, **mget** giúp hiển thị cửa sổ để người dùng tùy chọn những file cần upload/download.

§5.3 Quản lý phiên làm việc

- **ascii/binary**: Cho phép người dùng chuyển giữa hai chế độ truyền file thông qua tùy chọn giao diện.
- **status**: Hiển thị thông tin chi tiết phiên làm việc thông qua nút bấm *Status* bao gồm địa chỉ host, cổng kết nối, tên người dùng, chế độ truyền file, chế độ kết nối, thư mục hiện tại, tiến trình đang chạy.
- **passive**: Tùy chọn chuyển đổi giữa hai chế độ kết nối *active/passive* bằng checkbox *Passive*.
- **open/close**: Cho phép đăng nhập và kết nối vào tài khoản đã cấu hình trước đó ở FTP Server qua phần giao diện đăng nhập; người dùng có thể đóng kết nối bằng nút *Disconnect*.
- **quit**: Thoát khỏi chương trình, đóng các socket kết nối và giao diện.
- **help/?**: Hiển thị danh sách lệnh hỗ trợ trong cửa sổ Log để người dùng tham khảo nhanh.

§6 Hoạt động nhóm

§6.1 Đánh giá thành viên

Thành viên	Công việc	Mức độ hoàn thành
Nhật Huy	Code các thao tác trên file và folder với giao diện, quản lý trạng thái	100%
Duy Nhất	Code upload và download, quản lý trạng thái, quay video	100%
Khánh Linh	Viết tổng hợp file readme, báo cáo	100%

§6.2 Biên bản họp nhóm

§6.2.1 Họp nhóm ngày 24/06

Title: Thảo luận, xác nhận mục tiêu đồ án

Date: 24/06/2025

Time: 21:00 – 21:45

Location: Google Meet

Attendees	Purposes
Trịnh Khánh Linh Nguyễn Nhật Huy Vũ Duy Nhất	Đọc đề tài, tìm hiểu đồ án

Meeting Notes
- Tài liệu: Nội dung Secure FTP Client with Virus Scanning via ClamAVAgent (File) - Thảo luận về các yêu cầu của đồ án - Quyết định sử dụng Python là ngôn ngữ lập trình chính

Action Items	Take on	Due Date
Tìm hiểu, nắm được và hiểu cách vận hành, hoàn thiện đồ án	Cả nhóm	26/06

§6.2.2 **Họp nhóm ngày 26/06****Title:** Phân công công việc**Date:** 26/06/2025**Time:** 21:00 – 22:30**Location:** Google Meet

Attendees	Purposes
Trịnh Khánh Linh Nguyễn Nhật Huy Vũ Duy Nhất	Trao đổi, phân công công việc

Meeting Notes
• Trao đổi về các thông tin đã tự tìm hiểu được (cách cài đặt FileZilla, ClamAV,...) • Nói sơ lược về cách hoạt động của hệ thống • Phân công công việc chi tiết

Action Items	Take on	Due Date
Code các thao tác trên file và folder với giao diện	Nhật Huy	3-4/7
Code upload và download	Duy Nhất	1/7
Code quản lí trạng thái	Duy Nhất, Nhật Huy	1/7
Tổng hợp file Readme, Báo cáo	Khánh Linh	5/7
Quay video	Duy Nhất	5/7